RaftMessaging: a Proof of Concept of the Raft Consensus Algorithm

SOLDÀ MATTEO*, Università degli Studi di Padova - Dipartimento di Matematica, Italy

A fundamental problem in distributed computing is to achieve overall system reliability in presence of a number of faulty process. Consensus is a fundamental property to reach because it ensures data consistency. In this paper, we present RaftMessaging, a Raft-based Proof of Concept that shows an implementation of this consensus algorithm, designed to be easy to understand and develop, while providing strong fault tolerance. The focus of this work is to implement all the core features of the algorithm based on the original paper with some performance implementation in a Room-based Message Platform where users can choose their name, the gateway node and the room to write and read in.

CCS Keywords: Raft, Distributed Services, Fault Tolerance, Consensus Algorithm

ACM Reference Format:
Soldà Matteo. 2026. RaftMessaging: a Proof of Concept of the Raft Consensus Algorithm. In *Proceedings of Technical Report for RaftMessaging: a Proof of Concept of the Raft Consensus Algorithm.* ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The rise of large-scale cloud infrastructures and decentralized services has transitioned distributed systems from niche academic interests to the backbone of modern computing. These systems are characterized by their ability to provide high availability and scalability by dispersing workloads across a cluster of independent nodes. However, the shift toward distributed architectures introduces significant challenges, primarily centered on maintaining a consistent global state in the presence of partial network failures, message delays, and node crashes.

In concurrent and distributed runtimes, the core problem is reaching consensus, a process where a set of independent processes must agree on a single data value or a sequence of operations despite an unreliable environment. Historically, the Paxos algorithm was the industry standard for solving consensus. While mathematically robust, Paxos is notoriously difficult to understand and implement correctly in production environments, leading to a significant gap between theoretical models and practical engineering.

The Raft consensus protocol[2] was developed specifically to address this lack of understandability. Raft decomposes the complex problem of consensus into relatively independent subproblems, such as leader election, log replication, and safety. By establishing a strong leadership model, Raft ensures that only a single node, the Leader, is responsible for managing the replicated log and coordinating state transitions across the cluster. This hierarchical approach simplifies

Author's Contact Information: Soldà Matteo, matteo.solda.1@studenti.unipd.it, Università degli Studi di Padova - Dipartimento di Matematica, Padova, PD, Italy.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
This is the author's version of the work. It is posted here for your personal use. Not for redistribution.

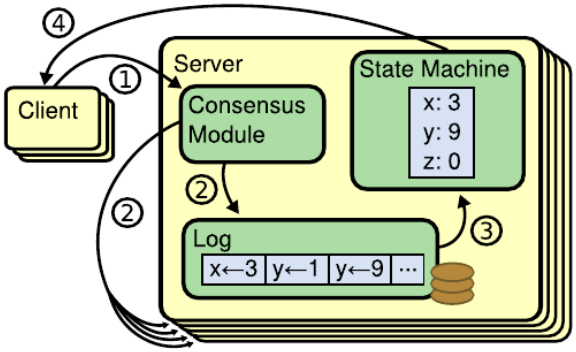

Fig. 1. Architecture of a Replicated State Machine, as described in the original Raft Paper[2]

the reasoning behind state consistency and allows for a more intuitive implementation of fault-tolerant systems.

This report explores the implementation of a Raft-based messaging system designed to leverage modern concurrent runtimes. By utilizing lightweight execution models such as Virtual Threads, the system aims to achieve high throughput and low latency while adhering to the strict safety properties defined by the Raft specification. The focus is placed on how the protocol manages log compaction through asynchronous snapshotting and maintains idempotency to provide "exactly-once" semantics in high-concurrency scenarios.

2 Raft Consensus Algorithm

The Raft consensus algorithm, introduced by Diego Ongaro and John Ousterhout, was designed to provide a highly understandable alternative to Paxos without compromising formal safety, liveness, or performance. In the context of distributed systems, reaching consensus is fundamentally equivalent to managing a Replicated State Machine. Raft achieves this by ensuring that all nodes in a cluster execute the same sequence of deterministic commands in the exact same order. To maximize comprehensibility and ease of implementation, Raft decomposes the complex consensus problem into three distinct and relatively independent subproblems:

- **Leader Election:** The robust mechanism by which a new leader is chosen when an existing leader fails or the cluster is first initialized. The leader acts as the single source of truth, exclusively handling client requests and orchestrating the replication process;
- **Log Replication:** The coordinated process through which the leader accepts operational commands from clients, such as dispatching messages to specific chat rooms, and securely propagates these log entries to follower nodes. This ensures that the state machine remains consistent across the entire distributed cluster;
- **Safety:** The strict algorithmic guarantee that if any server has applied a particular log entry to its state machine, no other server will ever apply a different command for that same log index. This property ensures that all future leaders will inherit all previously committed entries, maintaining immutable historical integrity.

To govern these operations, Raft enforces a strict lifecycle where every server in the cluster occupies one of three mutually exclusive states: *Leader*, *Follower*, or *Candidate*. Furthermore, time in Raft is divided into discrete, monotonically increasing logical intervals called *Terms*. Each term acts as a logical clock, allowing nodes to seamlessly detect and discard obsolete information, such as delayed messages from deposed leaders or partitioned nodes.

This is the author's version of the work. It is posted here for your personal use. Not for redistribution.

During normal operation, the cluster is commanded by exactly one leader, while all other nodes operate as passive followers. Followers do not initiate communication; they strictly respond to Remote Procedure Calls (RPCs) originating from leaders or candidates. In the specific context of the implemented group messaging system, the leader is the sole entity authorized to process incoming chat messages. It serializes these messages by appending them to the replicated log and orchestrates their distribution. To ensure non-blocking, scalable, and highly concurrent execution within the distributed runtime, inter-node communication is implemented via asynchronous RPCs.

2.1 Leader Election

Raft utilizes an active heartbeat mechanism to maintain cluster authority. A leader periodically broadcasts heartbeats (typically empty `AppendEntries` RPCs) to reset followers' internal timers and prevent unnecessary elections.

If a follower experiences network silence exceeding its *election timeout*, it assumes the leader is unreachable. It then increments its current logical term, transitions to the candidate state, casts a vote for itself, and dispatches `RequestVote` RPCs to the cluster.

A candidate remains in this state until it wins the election, another node establishes leadership, or the election times out. A candidate wins by receiving affirmative votes from a strict majority of the cluster for that term. Upon securing this quorum, it immediately assumes the leader role and broadcasts heartbeats to assert authority, forcing other candidates to step down.

A critical edge case in this process is the *Split Vote*. This occurs when multiple followers simultaneously become candidates, dividing the cluster's votes so that no node secures an absolute majority. To prevent indefinite deadlocks caused by repeated synchronized elections, Raft employs randomized election timeouts. By assigning each node a timeout duration from a fixed random interval, the algorithm breaks symmetry, ensuring one node will reliably time out first and secure leadership before its peers can intervene.

2.2 Log Replication

Each client request contains a command to be executed by the replicated state machines. The leader appends the command to its log as a new entry, then it issues `AppendEntries` RPCs in parallel to each of the servers in the cluster in order to replicate the command. If a follower node crashes or runs slowly, the leader reiterate the the `AppendEntries` RPC for that node.

Logs are stored with the command, the term and the index of that request. The term number is used to detect inconsistencies between logs.

An entry is called committed when the leader decides that is safe to apply a log entry to the state machines. Those entries are guaranteed to be durable and eventually executed by all the available state machines. A log entry is committed once the leader that created the entry has replicated it on a majority of the servers.

This management of the logs allows to ensures two properties that constitute the *Log Matching Property*:

- If two entries in different logs have the same index and term, they store the same command;
- If two entries in different logs have the same index and term, the logs are identical in all preceding entries.

This is the author's version of the work. It is posted here for your personal use. Not for redistribution.

In Raft, the leader handles inconsistencies by forcing the followers' logs to duplicate its own.

2.3 Safety

To guarantee consistent command execution across all state machines, Raft enforces strict restrictions on leader election and log commitment. Logs flow exclusively from leaders to followers, and leaders never overwrite their own entries.

Raft's *Leader Completeness Property* ensures that any committed entry will persistently reside in the logs of all future leaders. This invariant is maintained through an *up-to-date* log comparison during elections. To win, a candidate must secure a majority of votes. However, a voter will categorically reject any `RequestVote` RPC if its own log is more up-to-date (having a higher final log term, or a longer log if terms are equal) than the candidate's log.

Furthermore, a leader cannot safely commit entries from previous terms merely by counting replicas, as they might still be overwritten by future leaders. Instead, Raft restricts leaders to committing entries exclusively from their *current* term. By virtue of the *Log Matching Property*, successfully committing a current-term entry implicitly commits all preceding log entries.

The safety of this design is proven by contradiction. Assume an entry is committed in term T, but a leader lacking this entry is successfully elected in term U ($U > T$). Both the entry's commitment and the successful election require a majority quorum. Due to quorum intersection, at least one server must have accepted the entry in term T and later voted in term U. Since this overlapping server holds the committed entry (meaning its last log term is $\geq T$), its log is strictly more up-to-date than the candidate's (whose last log term $< T$). The server would therefore reject the candidate, rendering the election of a leader missing committed entries mathematically impossible.

2.4 Follower and Candidate Crashes

If a follower or a candidate crashes, then future `RequestVote` and `AppendEntries` RPCs sent to it will fail. Raft handles these failures by retrying indefinitely. If a leader crashes, the first follower to reach the timeout will start an election to be elected as leader for the new term.

2.5 Timing and Availability

While Raft's safety properties withstand timing anomalies (such as network latency or garbage collection pauses), its liveness and overall availability fundamentally depend on system timing. To ensure stable leadership and continuous progress, Raft mandates the following strict inequality:

$$\text{broadcastTime} \ll \text{electionTimeout} \ll \text{MTBF} \tag{1}$$

The terms are defined as follows:

- **broadcastTime:** The average time a server takes to dispatch parallel RPCs to the cluster and receive acknowledgments, encompassing both network latency and storage I/O.
- **electionTimeout:** The randomized interval a follower waits for leader heartbeats before initiating a new election.
- **MTBF (Mean Time Between Failures):** The average continuous operational uptime of a server before a crash.

The first inequality (broadcastTime $\ll$ electionTimeout) guarantees leadership stability. The broadcast time must be significantly smaller than the election timeout to ensure that an active leader can reliably transmit heartbeats. This prevents followers from timing out and triggering unnecessary election storms that would degrade cluster availability.

This is the author's version of the work. It is posted here for your personal use. Not for redistribution.

The second inequality (electionTimeout ≪ MTBF) ensures system responsiveness. When a leader inevitably crashes, the cluster is temporarily unavailable. Keeping the election timeout orders of magnitude smaller than the MTBF ensures that this leaderless downtime is infinitesimally small compared to normal operational uptime.

In practical deployments, *broadcastTime* typically ranges from 0.5 to 20 milliseconds. Consequently, *electionTimeout* is generally randomized between 150 and 300 milliseconds (which also efficiently resolves split votes). Given that modern MTBF is measured in months, this fundamental inequality is easily satisfied, allowing Raft to provide a highly available consensus mechanism.

2.6 Cluster Membership Changes

In dynamic distributed runtimes, the cluster configuration (the set of participating nodes) often needs to change to replace failed servers or adjust capacity. A naive, immediate transition from an old configuration to a new one can result in a split-brain scenario, where two disjoint majorities independently elect two different leaders for the same term.

To guarantee safety during these transitions, Raft employs a two-phase approach utilizing a transitional state called *joint consensus*. When a configuration change is initiated, the leader appends a joint configuration entry, combining the nodes of both the old and new configurations, to its log. While the system operates under joint consensus, any election or commitment requires a separate majority from both the old and the new configurations. Once this joint configuration is safely committed, the leader proposes the final new configuration. This staged approach ensures that disjoint majorities can never be formed, allowing the cluster to continue serving client requests seamlessly during the transition.

2.7 Log Compaction

In a practical distributed system, a replicated log cannot grow indefinitely without eventually exhausting storage capacity and prolonging startup times. Raft addresses this issue primarily through snapshotting, the simplest form of log compaction.

Instead of relying on a centralized coordinator, each server in Raft independently takes snapshots of its state machine once the committed log reaches a predefined size threshold. A snapshot captures the current state of the application alongside minimal routing metadata: the *last included index* and the *last included term*.

Once a snapshot is securely written to persistent storage, the server immediately discards all preceding log entries up to that index, reclaiming memory and disk space. In edge cases where a severely delayed follower requests log entries that the leader has already compacted, the leader resolves the discrepancy by sending an `InstallSnapshot` RPC. This mechanism effectively bounds the log size, minimizes node recovery time upon restart, and prevents network saturation when synchronizing slow nodes.

3 Technologies and Architectural Decisions

For the development of this Proof of Concept, Java 21 was selected primarily to leverage *Project Loom*'s Virtual Threads. In a distributed consensus protocol like Raft, nodes must concurrently manage numerous network connections; a leader, for instance, must broadcast parallel `AppendEntries` RPCs while simultaneously handling client requests.

This is the author's version of the work. It is posted here for your personal use. Not for redistribution.

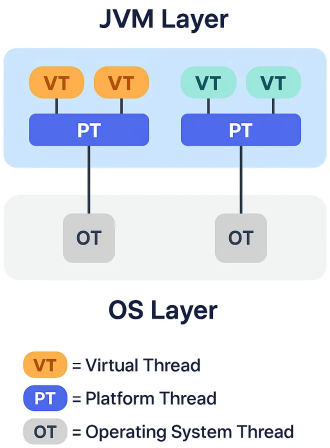

Fig. 2. Simplified Virtual Threads Architecture[1]

Virtual threads are lightweight entities managed directly by the JVM. Unlike traditional OS threads, they are multiplexed onto a small, bounded pool of carrier platform threads. When a virtual thread performs a blocking operation, such as network I/O or disk access, the JVM logically suspends it and immediately reassigns the underlying carrier thread to another ready virtual thread.

3.1 Implementation Analysis: Java 21 and Virtual Threads

This mechanism bypasses the costly overhead of OS-level context switching, allowing the application to concurrently manage millions of virtual threads. For this Raft implementation, it enables a straightforward *thread-per-request* approach. A new thread is dynamically allocated for every asynchronous RPC without requiring complex reactive frameworks or callbacks. Consequently, the source code maintains a sequential, synchronous logical flow, which greatly simplifies debugging and state management. Furthermore, Java's rich ecosystem of concurrent libraries (e.g., `ConcurrentHashMap`, `ReentrantLock`) provides robust primitives for safely managing access to the replicated log and the State Machine.

3.2 Limitations and Challenges for Consensus Systems

Despite these architectural advantages, relying on the JVM introduces specific structural vulnerabilities for a strictly time-dependent algorithm like Raft:

- **Garbage Collection Pauses:** "Stop-The-World" GC cycles can temporarily suspend all threads. If a pause exceeds the *election timeout*, a follower might incorrectly infer leader failure, triggering unnecessary *election storms* that degrade cluster stability.

This is the author's version of the work. It is posted here for your personal use. Not for redistribution.

- **Thread Pinning:** In Java 21, using native `synchronized` blocks or JNI code "pins" the virtual thread to its OS carrier thread. During intensive I/O operations, such as flushing the Write-Ahead Log or Snapshots to disk, this behavior blocks the entire hardware thread, creating severe performance bottlenecks.

3.3 Alternative Programming Languages

To mitigate JVM-induced latency unpredictability, industrial distributed systems often employ languages offering deterministic hardware control. **Rust** (e.g., TiKV) enforces memory management at compile time, completely eliminating GC pauses and ensuring election timeouts are triggered solely by actual network failures. Alternatively, **Go** (e.g., Consul, etcd) provides *Goroutines*, a highly optimized concurrency model conceptually identical to virtual threads, paired with an ultra-low latency GC and a scheduler that handles blocking I/O transparently without suffering from thread pinning.

3.4 System Architecture and Package Structure

The implemented Raft consensus algorithm utilizes a modular architecture that strictly decouples the state machine, network communication, and persistent storage across three functional packages:

- **`com.raft.core`:** Provides fundamental domain entities and storage abstractions. Key components include the `Role` enumeration and the `LogEntry` class, which embeds sequence numbers to guarantee exactly-once semantics. The persistence layer is managed by `WalStorage` (using Write-Ahead Logging for crash durability) and `Snapshot` (for memory compaction). The `Network` interface abstracts the transport layer via `InMemoryNetwork` or `HttpNetwork`.
- **`com.raft.rpc`:** Defines the immutable data structures for inter-node communication. It includes standard RPCs (`AppendEntries`, `RequestVote`, `InstallSnapshot`) and modern optimizations like `PreVote`, which prevents partitioned nodes from artificially inflating terms upon reconnection.
- **`com.raft.node`:** Centers around the `Node` class, which implements the complete state machine logic using Java 21 Virtual Threads for massive parallelism. Critical operations include `propose` (client entry and deduplication), `updateCommitIndex` (quorum validation and commitment), `takeSnapshot` (asynchronous, non-blocking log compaction), and `handleAppendEntries` (strict safety and Log Matching verification).

3.5 Network Communication and Client Integration

The `HttpNetwork` class bridges Raft's internal consensus logic with the external environment, facilitating peer-to-peer communication and web client integration.

3.5.1 The `HttpNetwork` *Component.* Leveraging Java's native HTTP libraries, this component operates in two primary roles:

- **Embedded HTTP Server:** Exposes dedicated URL endpoints for internal Raft RPCs (`/requestVote`, `/appendEntries`, etc.) and public client interactions (`/clientPropose`, `/clientGet`).
- **Asynchronous HTTP Client:** Routes outgoing RPCs using Java's asynchronous `HttpClient`. Combined with Virtual Threads, this enables non-blocking, parallel heartbeat dispatch across the cluster, eliminating network latency bottlenecks.

3.5.2 Messaging Platform Simulation via `index.html`. A Single Page Application (`index.html`) abstracts the replicated state machine into a room-based chat interface via two main flows:

This is the author's version of the work. It is posted here for your personal use. Not for redistribution.

- **Message Dispatch (`/clientPropose`):** The client sends an HTTP POST formatted as `SEND <room> [<user>]: <message>`. If received by the Leader, the command is replicated and returns a 200 OK upon quorum. Followers reject the request, requiring the client to try an alternative gateway.
- **Real-Time Synchronization (`/clientGet`):** The client asynchronously polls the node every second. Thanks to Raft's log consistency, any queried node returns a coherent chat history, simulating a centralized backend.

As the primary focus of this Proof of Concept is the consensus engine, explicit security mechanisms (e.g., authentication, TLS) are omitted; consequently, endpoints remain vulnerable to unauthorized `GET` or `POST` requests via tools like *curl*.

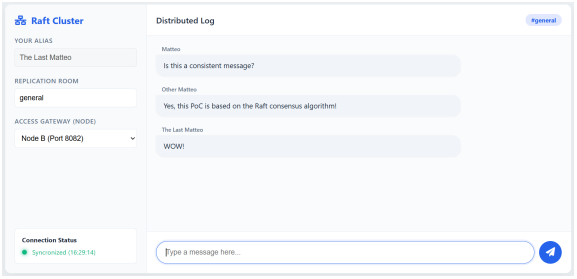

Fig. 3. PoC Web Interface

4 Discussion

While the implemented system rigorously adheres to the foundational Raft algorithm, it incorporates several critical optimizations and modern architectural paradigms necessary for production-grade distributed systems.

To prevent partitioned nodes with artificially inflated terms from disrupting the cluster upon reconnection, the system implements the **Pre-Vote algorithm**. Before transitioning to the `CANDIDATE` state, a follower must gather a Pre-Vote quorum by broadcasting a `PreVoteRequest`. Peers approve only if the requester's log is up-to-date and no valid leader heartbeat was recently received.

To prevent stale reads during network partitions without the heavy I/O cost of routing reads through the log, the system uses the **ReadIndex optimization**:

(1) The leader records its `commitIndex` as `readIndex`.
(2) It broadcasts empty `AppendEntries` heartbeats to prove active cluster authority.
(3) It blocks the read operation until its `lastApplied` index catches up to the `readIndex`.
(4) The request is served directly from the in-memory `stateMachine`.

To provide strict **exactly-once semantics** and prevent duplicate commits during network retries, the `Node` maintains a `clientSession` map. Each `LogEntry` carries a `clientID` and a strictly increasing `sequenceNum`. Commands with sequence numbers less than or equal to the highest processed number for that client are safely ignored.

Instead of the complex Joint Consensus, the system implements Ongaro's **Single-Server Membership Change**. The protocol adds or removes exactly one node at a time. If a `CONF_ADD_SERVER` or `CONF_REMOVE_SERVER` command is currently uncommitted, subsequent membership requests are rejected. This guarantees overlapping majorities and prevents split-brain scenarios.

Beyond algorithmic extensions, the implementation introduces significant runtime optimizations:

This is the author's version of the work. It is posted here for your personal use. Not for redistribution.

- **Virtual Threads (Project Loom):** The network abstraction assigns a lightweight Virtual Thread to every outgoing RPC. This eliminates complex asynchronous callbacks, minimizes context-switching, and allows the system to achieve throughputs exceeding 15,000 requests per second.
- **Natural RPC Batching:** Instead of sending one command per RPC, the leader dynamically aggregates all pending `LogEntry` objects into a single `AppendEntriesRequest` array, significantly optimizing network utilization under high stress.

System robustness and adherence to Raft invariants were validated through automated test suites simulating node failures, split-votes, and network partitions. To evaluate architectural efficiency, stress tests subjected the cluster to continuous loads of 15,000 to 150,000 requests.

Table 1 reports the average processing time per single request across varying node counts and client concurrency. These metrics highlight how the system maintains optimal operational latencies by leveraging virtual thread multiplexing, effectively mitigating the overhead typical of traditional OS-thread architectures.

Nodes/Clients	10	50	100	200
3	12.0	12.3	14	14.9
5	12.3	13.9	15.3	16.5
7	13.9	15.4	16.7	18.1
15	15.6	16.9	18.4	20.01

Table 1. Average time to correctly replicate a single request coming from a variable number of concurrent clients and a variable number of nodes

5 Future Work

While the Java 21 Virtual Threads implementation successfully demonstrates a viable Raft-based messaging system, future iterations should focus on the following architectural optimizations for production readiness:

- **Mitigating Thread Pinning:** Replacing native `synchronized` blocks with `ReentrantLock` to ensure virtual threads can yield during Write-Ahead Log (WAL) disk I/O, maximizing concurrency limits.
- **Transport Optimization:** Transitioning from HTTP/1.1 to a binary-packed protocol like gRPC over HTTP/2 or a custom TCP protocol to minimize inter-node RPC latency and serialization overhead.
- **Snapshot Chunking:** Transmitting `InstallSnapshot` payloads in smaller, sequential byte blocks to prevent heap memory exhaustion and network timeouts when syncing massive datasets.
- **Security and Authentication:** Integrating mutual TLS (mTLS) for secure inter-node communication and JWT-based authentication at the HTTP gateway for secure client interactions.

6 Conclusion

The development and evaluation of this Proof of Concept have successfully demonstrated the practical viability and inherent comprehensibility of the Raft consensus algorithm. Historically, distributed consensus has been considered a notoriously complex domain, often resulting in implementations that significantly diverge from their theoretical foundations. In contrast, Raft's intentional decomposition of the consensus problem into distinct, manageable subproblems, as leader election, log replication, and safety, makes it remarkably straightforward to apply to real-world scenarios.
This project proved that a fully functional, fault-tolerant Replicated State Machine can be built without sacrificing code

This is the author's version of the work. It is posted here for your personal use. Not for redistribution.

readability or architectural simplicity. By applying Raft to a distributed messaging platform, it was demonstrated how strict data consistency and exactly-once semantics can be guaranteed across a cluster, even in the presence of severe network partitions or node failures. Furthermore, the integration of Java 21's Virtual Threads showcased how modern concurrency models can perfectly complement Raft's synchronous RPC requirements. This approach allowed the system to achieve massive network fan-out and high throughput while avoiding the cognitive overhead and codebase fragmentation typical of reactive programming paradigms.

The complete source code for this project is open-source and publicly accessible to review the implementation details, inspect the comprehensive test suites, and deploy the cluster locally. The provided configuration allows users to easily launch the system, observe the leader election process in real-time through the web interface, and verify the algorithm's resilience firsthand.

Project Repository: https://github.com/matnut2/RaftMessaging

References

[1] Kocak Burak. [n. d.]. https://medium.com/@burakkocakeu/harnessing-project-loom-for-lightweight-threads-in-java-0c7aef119327

[2] Diego Ongaro and John Ousterhout. 2014. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference* (Philadelphia, PA) *(USENIX ATC'14)*. USENIX Association, USA, 305–320.

Received March 19, 2026; revised NA; accepted NA

This is the author's version of the work. It is posted here for your personal use. Not for redistribution.